



Experiment 8

AIM: Use Postman to test sample API.

1. Objective

- Understand how to test REST APIs using Postman by sending different types of HTTP requests.
 - Get
 - Post
 - Put
 - Delete

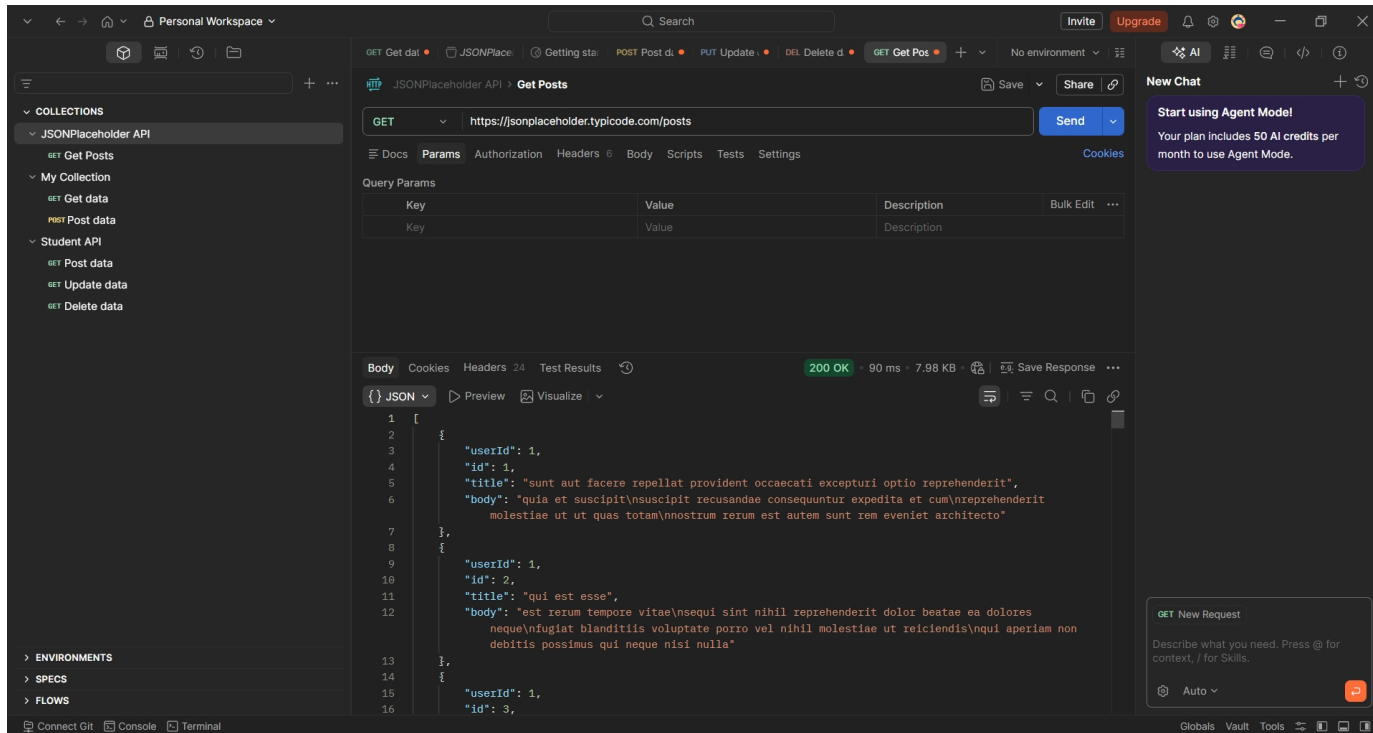
2. Prerequisites

- postman
- API to be tested
 - Create a simple node.js API / use some free fake API

3. Test API

Get:

URL: <https://jsonplaceholder.typicode.com/posts>



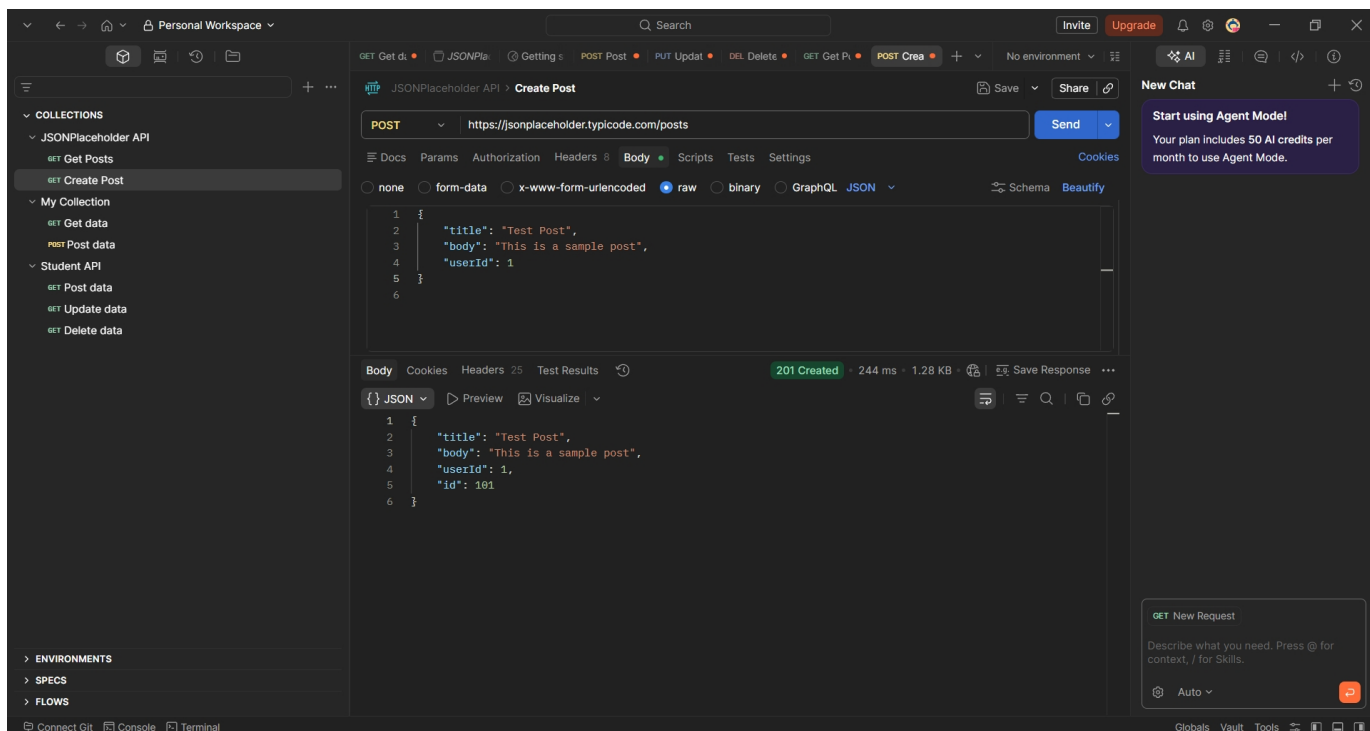
The screenshot shows a REST client interface with a sidebar on the left containing a 'COLLECTIONS' list. The main area displays a GET request to 'https://jsonplaceholder.typicode.com/posts'. The response is a 200 OK status with a 90 ms response time and a 7.98 KB body. The response body is a JSON array of 3 posts, each with 'userId', 'id', 'title', and 'body' fields. The interface also includes a 'New Chat' panel on the right and a 'GET New Request' panel at the bottom right.

```
1 [
2   {
3     "userId": 1,
4     "id": 1,
5     "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6     "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
7   },
8   {
9     "userId": 1,
10    "id": 2,
11    "title": "qui est esse",
12    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
13  },
14  {
15    "userId": 1,
16    "id": 3,
```

Post:

`https://jsonplaceholder.typicode.com/posts`

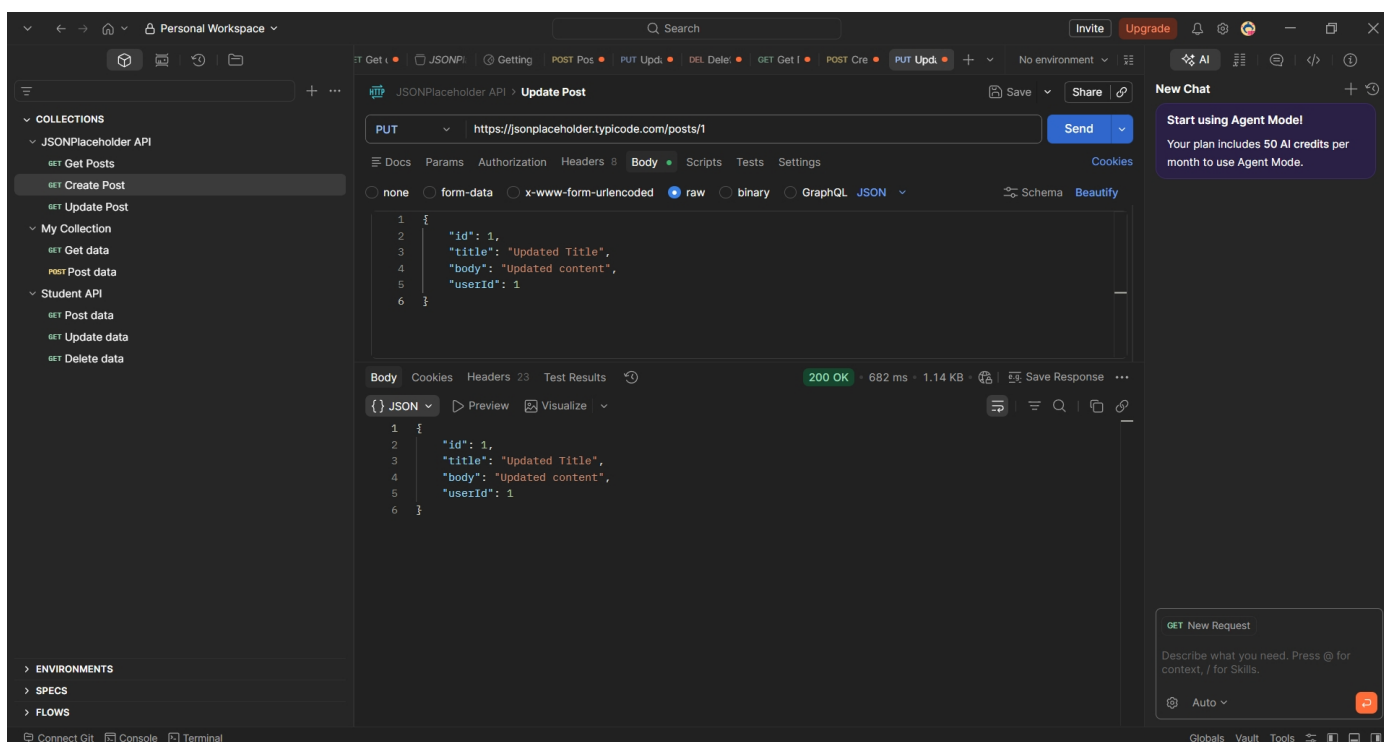
```
{  
  
  "title": "Test Post",  
  
  "body": "This is a sample post",  
  
  "userId": 1  
}
```



Put:

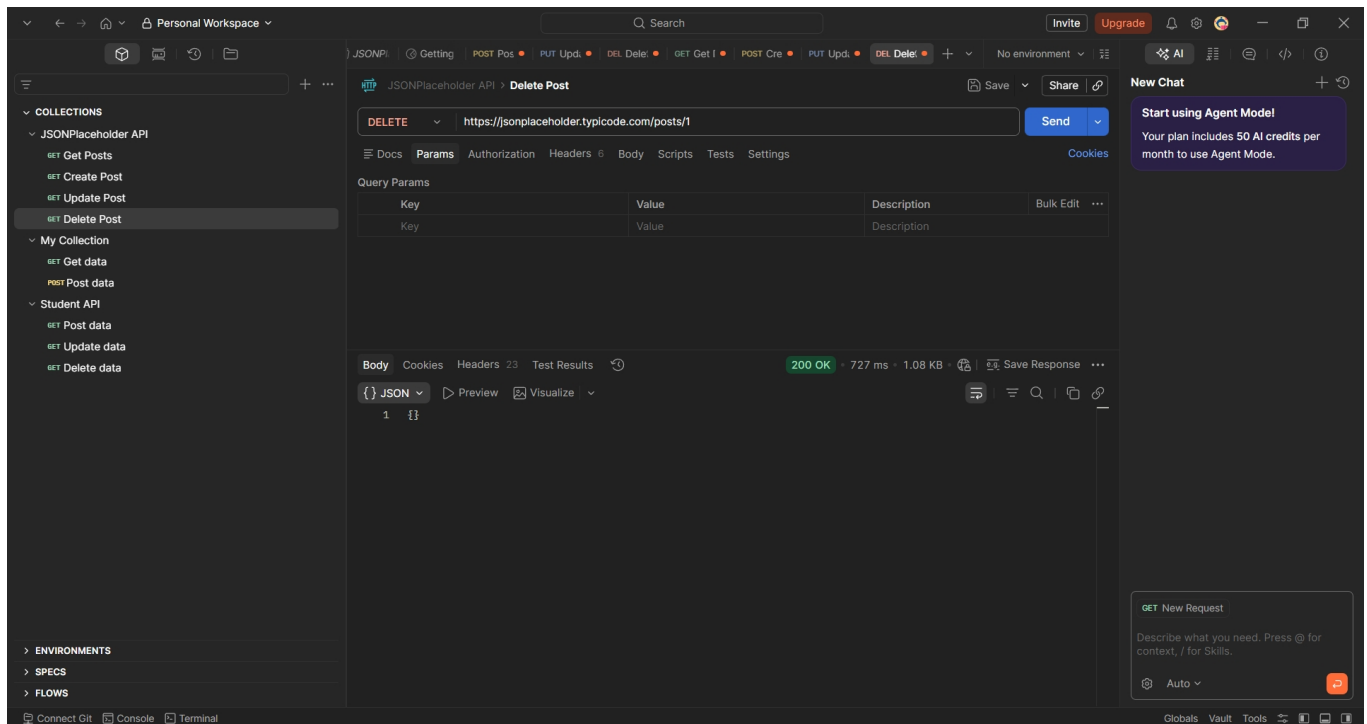
`https://jsonplaceholder.typicode.com/posts/1`

```
{  
  
  "id": 1,  
  
  "title": "Updated Title",  
  
  "body": "Updated content",  
  
  "userId": 1  
}
```



Delete:

<https://jsonplaceholder.typicode.com/posts/1>



4. Create node.js API

Step 1: Create Project

```
mkdir simple-api
```

```
cd simple-api
```

```
npm init -y
```

```
npm install express
```

Step 2: Create server.js file

```
const express = require('express');
```

```
const app = express();
```

```
app.use(express.json());
```



```
let students = [

  { id: 1, name: "Viraj", branch: "CE" },

  { id: 2, name: "Ravi", branch: "CE" }

];

app.get('/students', (req, res) => {

  res.json(students);

});

app.get('/students/:id', (req, res) => {

  const student = students.find(s => s.id == req.params.id);

  res.json(student);

});

app.post('/students', (req, res) => {

  const newStudent = {

    id: students.length + 1,

    name: req.body.name,

    branch: req.body.branch

  };

  students.push(newStudent);

  res.status(201).json(newStudent);

});

app.put('/students/:id', (req, res) => {

  const student = students.find(s => s.id == req.params.id);

  if (student) {

    student.name = req.body.name;

    student.branch = req.body.branch;
```



```
res.json(student);

} else {

res.status(404).json({ message: "Student not found" });

}

});

// DELETE - Remove student

app.delete('/students/:id', (req, res) => {

students = students.filter(s => s.id !== req.params.id);

res.json({ message: "Student deleted" });

});

app.listen(3000, () => {

console.log("Server running on http://localhost:3000");

});
```

Step 3: Run server

Node server.js

Command :

Microsoft Windows [Version 10.0.26200.8037]

(c) Microsoft Corporation. All rights reserved.

C:\Users\prash>cd C:\Users\prash\Desktop

C:\Users\prash\Desktop>mkdir simple-api

C:\Users\prash\Desktop>cd simple-api

C:\Users\prash\Desktop\simple-api>npm init -y

Wrote to C:\Users\prash\Desktop\simple-api\package.json:

```
{
```



```
"name": "simple-api",  
  
"version": "1.0.0",  
  
"description": "",  
  
"main": "index.js",  
  
"scripts": {  
  "test": "echo \"Error: no test specified\" && exit 1"  
},  
  
"keywords": [],  
  
"author": "",  
  
"license": "ISC",  
  
"type": "commonjs"  
}
```

```
C:\Users\prash\Desktop\simple-api>npm install express
```

```
added 65 packages, and audited 66 packages in 2s
```

```
22 packages are looking for funding
```

```
run `npm fund` for details
```

```
found 0 vulnerabilities
```

```
C:\Users\prash\Desktop\simple-api>notepad server.js
```

```
Server running on http://localhost:3000
```

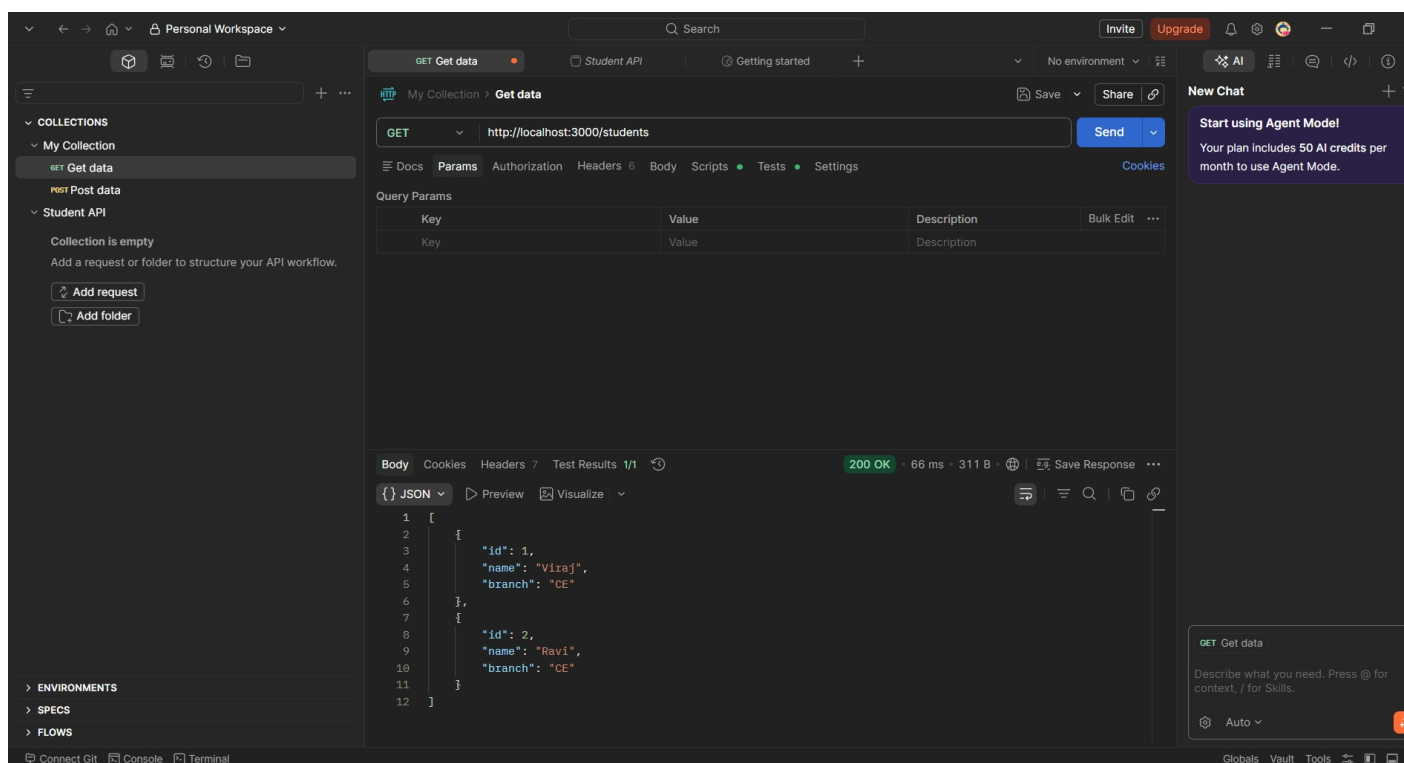

5. Procedure TEST using postman

1. GetAll:

Type: Get

URL : `http://localhost:3000/students`

Check response



2. Post:

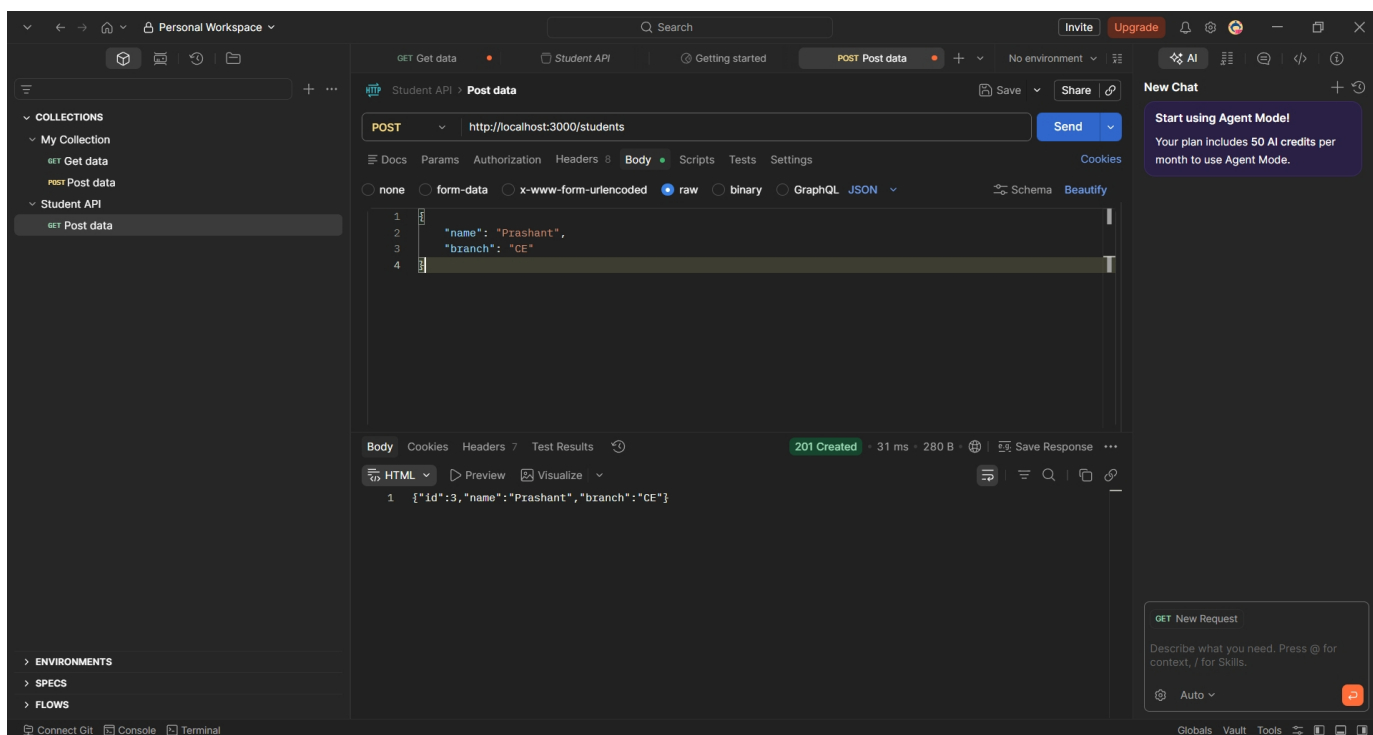
Type: Post

URL: `http://localhost:3000/students`

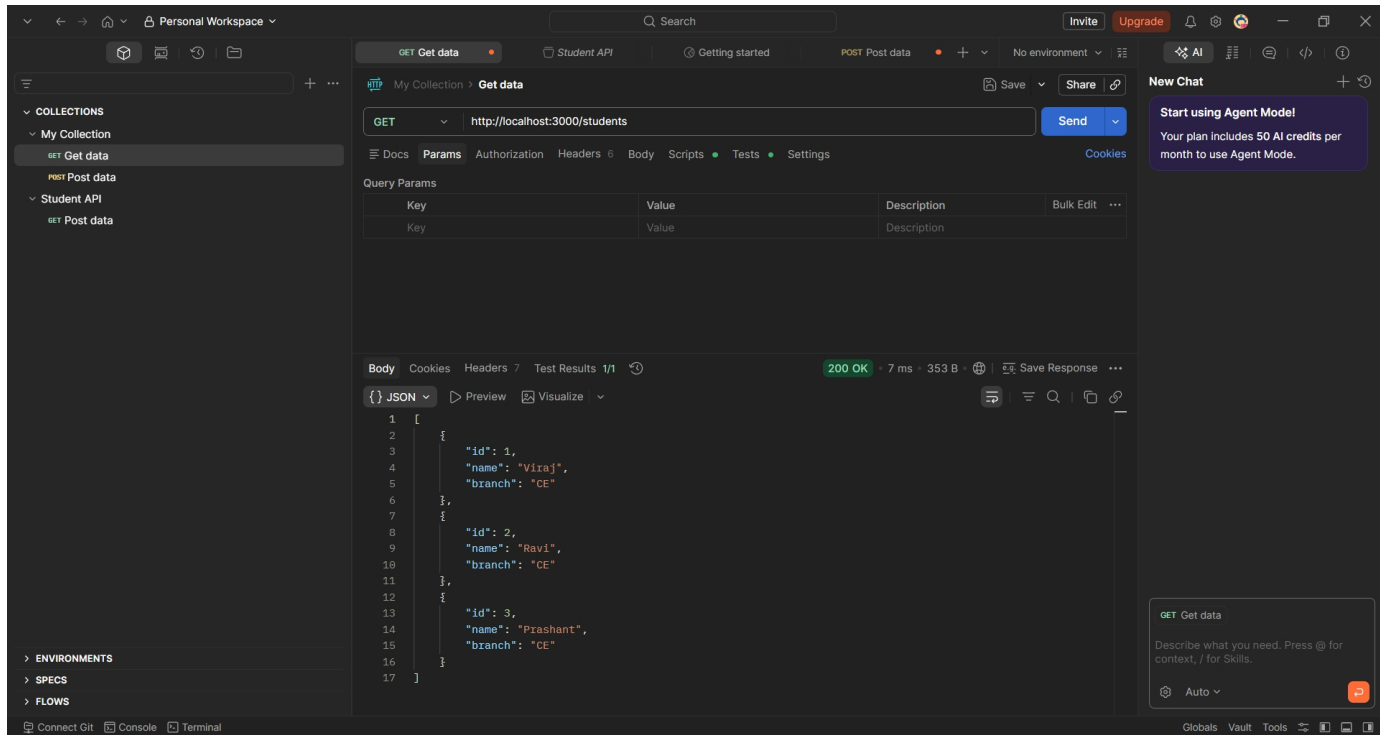
Body:

```
{  
  "name": "Prashant",  
  "branch": "CE"  
}
```

Check response



Send Get request to see new data inserted



The screenshot shows the Postman application interface. On the left, the 'COLLECTIONS' panel is visible, showing a collection named 'My Collection' with two items: 'GET Get data' and 'POST Post data'. The 'GET Get data' item is selected. The main workspace displays the details of the 'GET Get data' request. The URL is 'http://localhost:3000/students'. The response is a 200 OK status with a response time of 7 ms and a size of 353 B. The response body is a JSON array of three student objects, displayed in the 'Body' tab. The JSON data is as follows:

```
[{"id": 1, "name": "Viraj", "branch": "CE"}, {"id": 2, "name": "Ravi", "branch": "CE"}, {"id": 3, "name": "Prashant", "branch": "CE"}]
```

On the right side of the interface, there is a 'New Chat' panel with a message: 'Start using Agent Mode! Your plan includes 50 AI credits per month to use Agent Mode.' Below this, there is a chat input field with the text 'GET Get data' and a prompt: 'Describe what you need. Press @ for context, / for Skills.' The bottom of the interface shows a status bar with 'Connect Git', 'Console', and 'Terminal' buttons.

3. Put request:

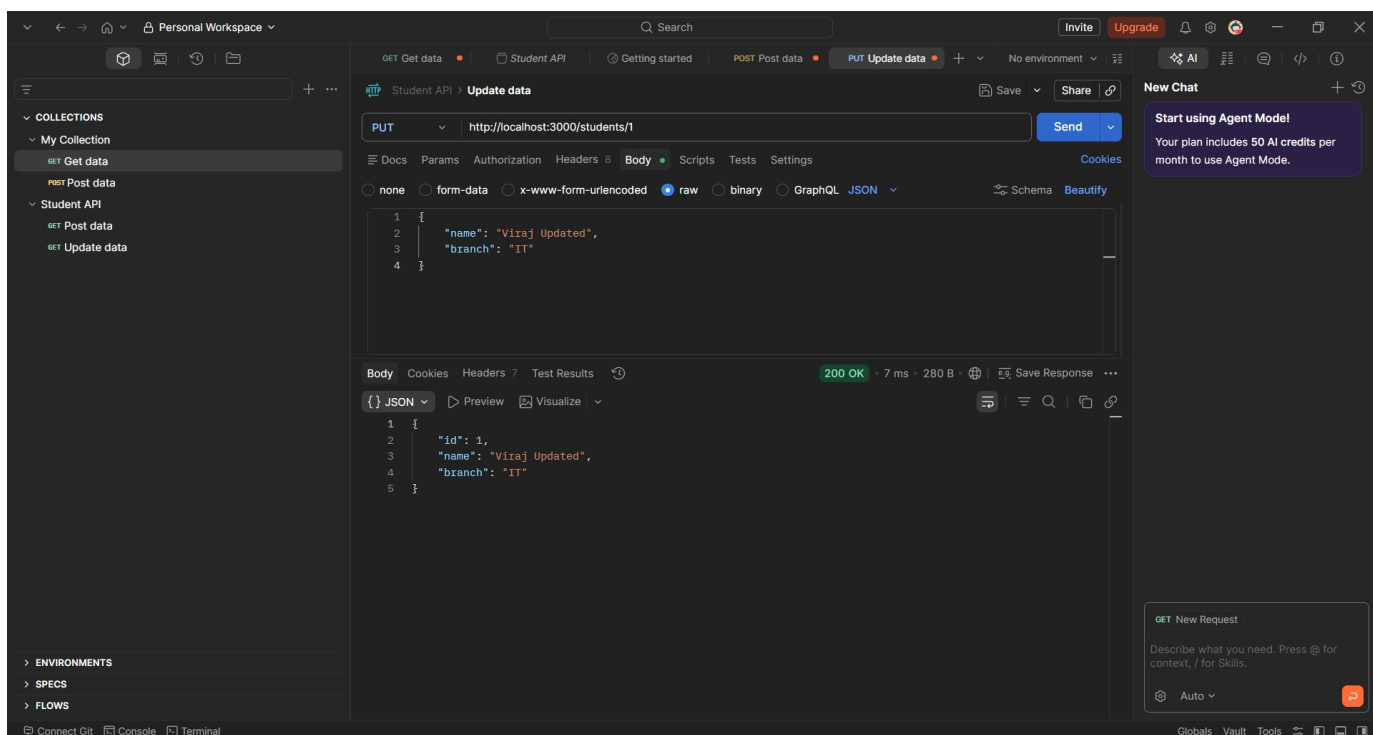
Type: PUT

URL : `http://localhost:3000/students/1`

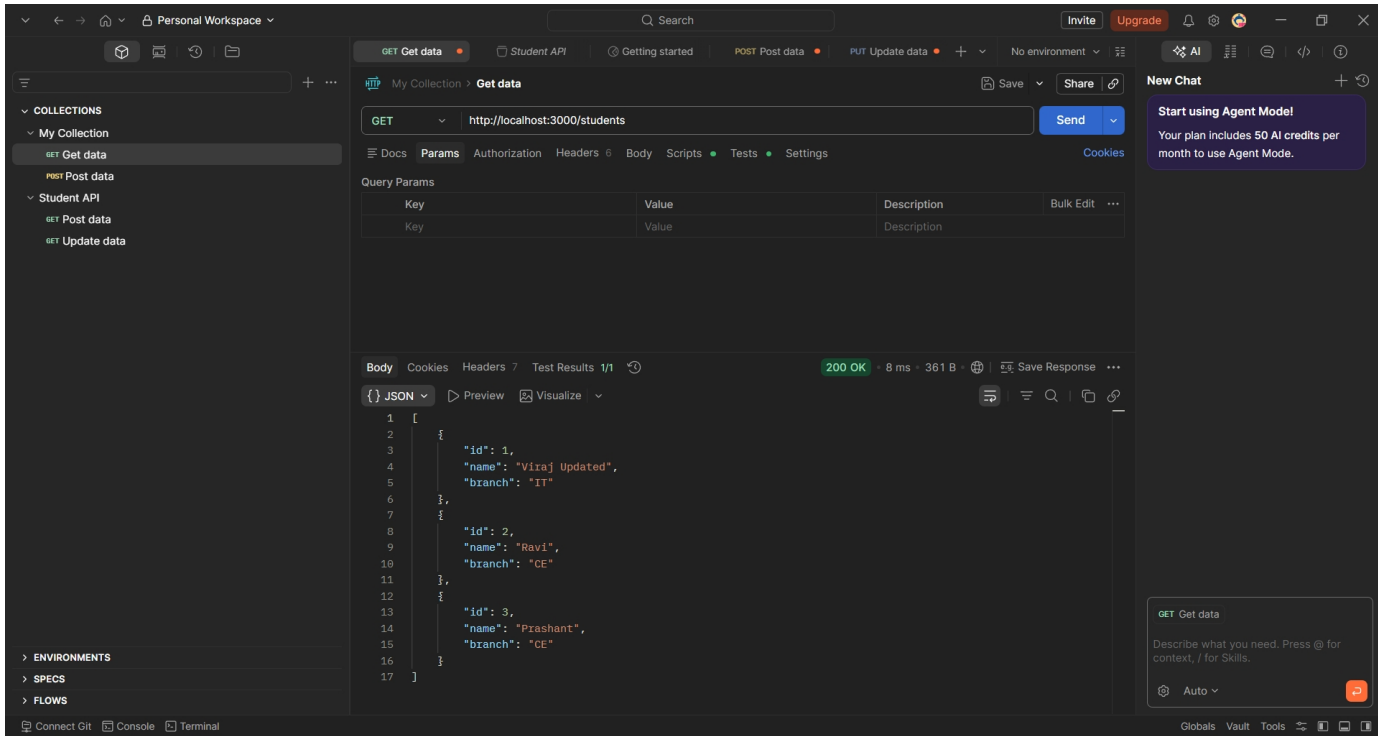
Body:

```
{  
  "name": "Viraj Updated",  
  "branch": "IT"  
}
```

Check response



Send Get request to see updated data



The screenshot shows the Postman interface with a GET request to `http://localhost:3000/students`. The response is a 200 OK status with a response time of 8 ms and a body size of 361 B. The response body is displayed in JSON format, showing an array of three student objects.

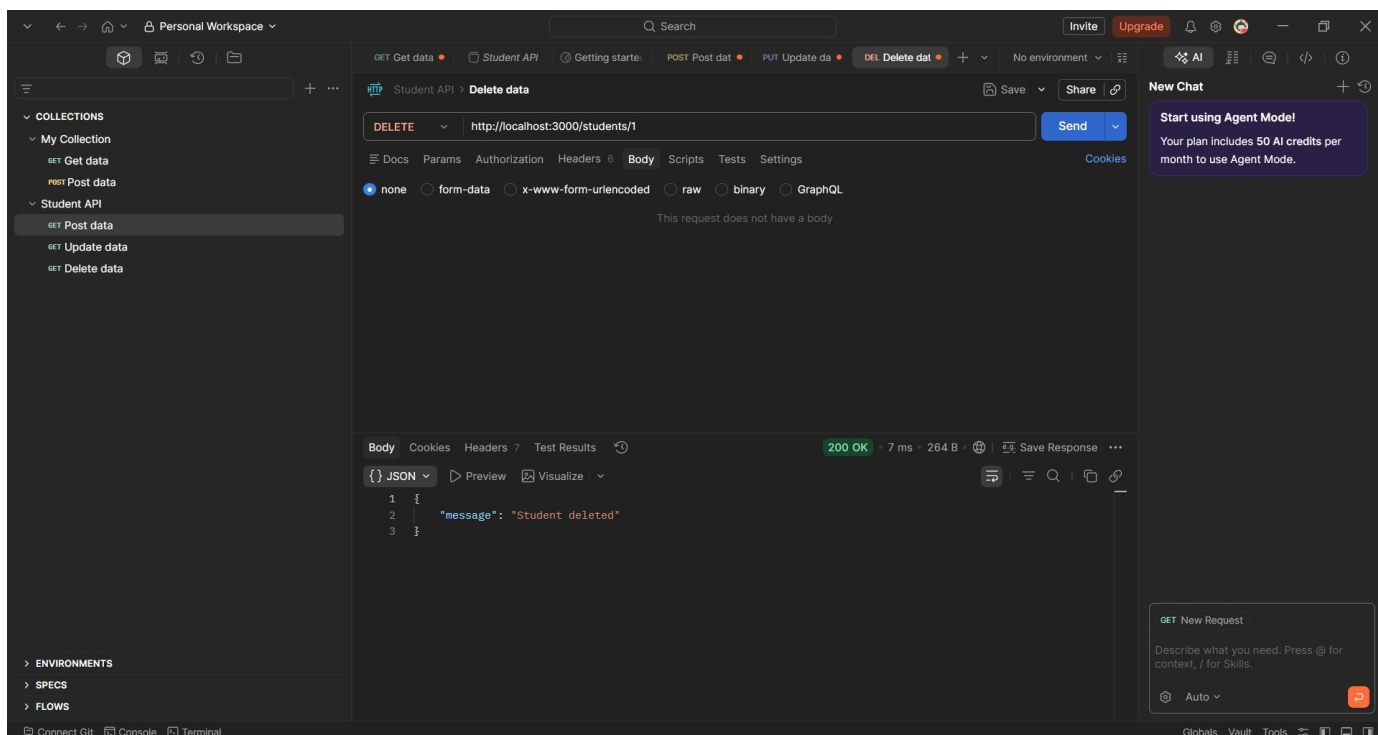
```
1 [
2   {
3     "id": 1,
4     "name": "Viraj Updated",
5     "branch": "IT"
6   },
7   {
8     "id": 2,
9     "name": "Ravi",
10    "branch": "CE"
11  },
12  {
13    "id": 3,
14    "name": "Prashant",
15    "branch": "CE"
16  }
17 ]
```

4. Delete request:

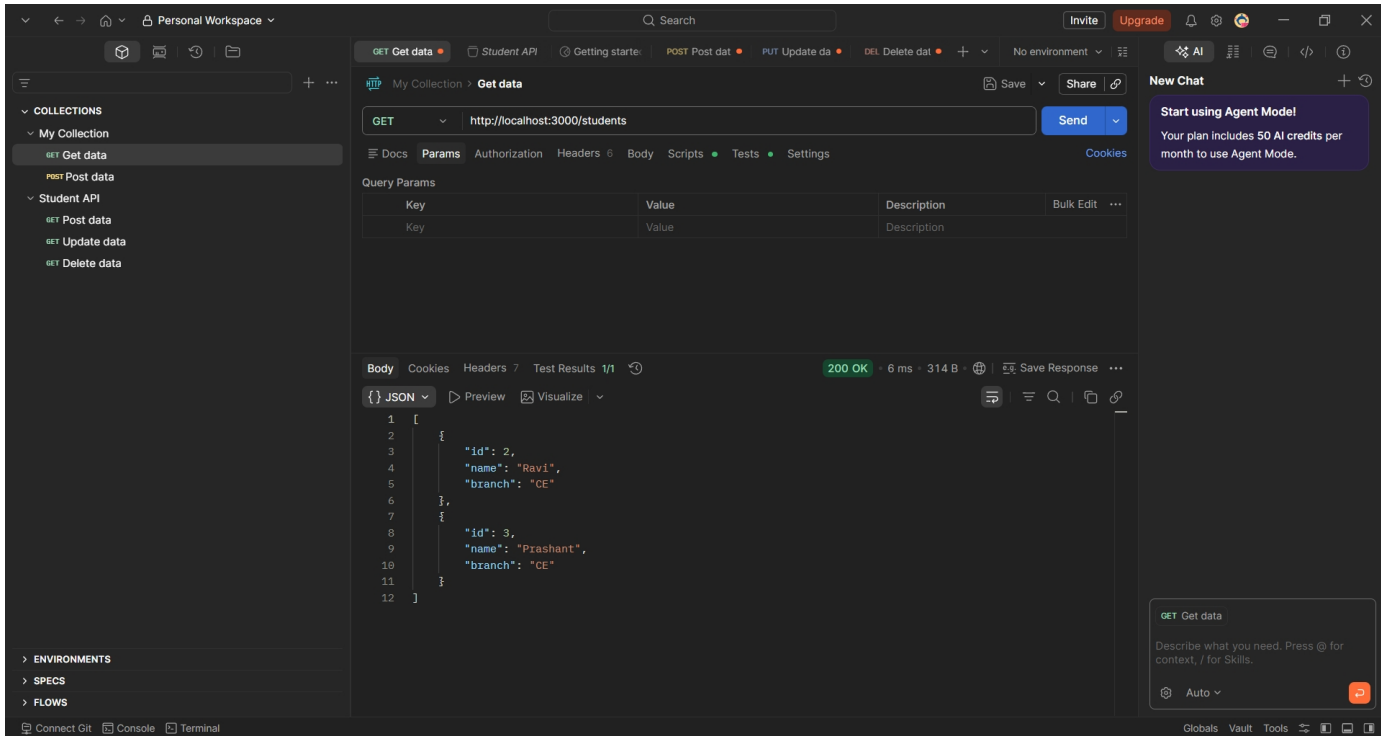
Type: DELETE

URL : `http://localhost:3000/students/1`

Check response



Send Get request to see updated data



The screenshot shows the Postman application interface. On the left, the 'My Collection' is expanded, showing a 'GET Get data' request. The main panel displays the details of this request, including the URL 'http://localhost:3000/students' and the 'Send' button. Below the URL bar, the 'Query Params' table is empty. The 'Body' tab is selected, showing a JSON response with a status of '200 OK'. The response body is a JSON array containing two student objects.

Key	Value	Description	Bulk Edit
Key	Value	Description	

```

1  [
2    {
3      "id": 2,
4      "name": "Ravi",
5      "branch": "CE"
6    },
7    {
8      "id": 3,
9      "name": "Prashant",
10     "branch": "CE"
11   }
12 ]
  
```

On the right side, there is a 'New Chat' panel with a message: 'Start using Agent Model! Your plan includes 50 AI credits per month to use Agent Mode.' At the bottom right, there is a 'GET Get data' chat window with a prompt: 'Describe what you need. Press @ for context / for Skills.'